

Chapter 13

Linearized Kalman Filtering

Introduction

SO FAR we have seen how linear Kalman filters are designed and how they perform when the real world can be described by linear differential equations expressed in state-space form and when the measurements are linear functions of the states. In addition, we have seen how extended Kalman filtering can be used when the real world is either described by nonlinear differential equations or measurements are not linear functions of the filter states. With both the linear and extended Kalman filters, no a priori assumption was made concerning additional information that might be available. The only information required to start the extended Kalman filter was the initial state estimates and an initial covariance matrix.

In some problems, such as those involving satellite tracking, a great deal of additional a priori information exists. For example, in satellite tracking we know in advance the approximate orbit of the satellite. In principle, this type of information should make the overall filtering problem easier. In this chapter two examples will be presented showing how a linearized Kalman filter can be built if nominal trajectory information exists for a nonlinear problem. In addition, we will also see what happens to the performance of the linearized Kalman filter if the nominal trajectory information is not accurate.

Theoretical Equations¹

To apply linearized Kalman filtering techniques, it is first necessary to describe the real world by a set of nonlinear differential equations. We have already seen from the chapters on extended Kalman filtering that these equations also can be expressed in nonlinear state-space form as a set of first-order nonlinear differential equations or

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{w}$$

where $\mathbf{x}$ is a vector of the system states, $\mathbf{f}(\mathbf{x})$ is a nonlinear function of those states, and $\mathbf{w}$ is a random zero-mean process. The process-noise matrix describing the random process $\mathbf{w}$ for the preceding model is giving by

$$\mathbf{Q} = E(\mathbf{w}\mathbf{w}^T)$$

The measurement equation, required for the application of a linearized Kalman filter, is considered to be a nonlinear function of the states according to

$$z = h(\mathbf{x}) + \mathbf{v}$$

where $\mathbf{v}$ is the measurement noise, which is considered to be a zero-mean uncorrelated random process described by the measurement noise matrix $\mathbf{R}$, which is defined as

$$\mathbf{R} = E(\mathbf{v}\mathbf{v}^T)$$

If a nominal trajectory for this nonlinear problem exists, then we can say that there is also an additional set of nonlinear differential equations. We can write the nonlinear differential equations describing the trajectory in compact form as

$$\dot{\mathbf{x}}_{\text{NOM}} = f(\mathbf{x}_{\text{NOM}})$$

where $\mathbf{x}_{\text{NOM}}$ represents the states of the nominal trajectory. We can now develop an error equation that is the difference between the states of the actual trajectory $\mathbf{x}$ and the states of the nominal trajectory $\mathbf{x}_{\text{NOM}}$ and is given by

$$\Delta \mathbf{x} = \mathbf{x} - \mathbf{x}_{\text{NOM}}$$

where $\Delta \mathbf{x}$ represent error states. We can then write an approximate linearized differential equation for the error states (i.e., difference between actual states and nominal trajectory states) as

$$\Delta \dot{\mathbf{x}} = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{NOM}}} \Delta \mathbf{x} + \mathbf{w}$$

The preceding equation is of the same form as the extended Kalman-filtering state-space equation, except we are now working with error states and the partial derivative matrix is evaluated on the nominal trajectory, rather than at the current state estimate.

In a similar way we can also develop an error equation for the measurement. The error measurement is the difference between the actual measurement and the nominal measurement or

$$\Delta z = z - z_{\text{NOM}}$$

Therefore, the error measurement equation is given by

$$\Delta z = \left. \frac{\partial h(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{NOM}}} \Delta \mathbf{x} + \mathbf{v}$$

Again, as was the case before, the preceding linearized error measurement equation is of the same form as the extended Kalman-filtering linearized

measurement equation, except we are now working with error states and the partial derivative matrix is evaluated on the nominal trajectory, rather than at the projected state estimate.

From the preceding error equations we can define the systems dynamics matrix and the measurement matrix to be matrices of partial derivatives or

$$\mathbf{F} = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{NOM}}}$$

$$\mathbf{H} = \left. \frac{\partial h(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{NOM}}}$$

For systems in which the measurements are discrete, we can rewrite the measurement equation as

$$\Delta \mathbf{z}_k = \mathbf{H} \Delta \mathbf{x}_k + \mathbf{v}_k$$

The fundamental matrix, which is required for discrete systems, can be obtained directly from the systems dynamics matrix. We have already shown in preceding chapters that the fundamental matrix can be approximated by the Taylor-series expansion for $\exp(\mathbf{F}T_s)$ and is given by

$$\Phi_k = \mathbf{I} + \mathbf{F}T_s + \frac{\mathbf{F}^2 T_s^2}{2!} + \frac{\mathbf{F}^3 T_s^3}{3!} + \dots$$

where T_s is the sample time and $\mathbf{I}$ is the identity matrix. As was shown in the chapters pertaining to extended Kalman filtering, the Taylor series is often approximated by only the first two terms or

$$\Phi_k \approx \mathbf{I} + \mathbf{F}T_s$$

The measurement and fundamental matrices of the linearized Kalman filter may be considered to be linear functions of time because they can be computed offline based on information from the nominal trajectory. The Riccati equations, required for the computation of the Kalman gains, are identical to those of the linear and extended Kalman filters and are repeated here for convenience.

$$\mathbf{M}_k = \Phi_k \mathbf{P}_{k-1} \Phi_k^T + \mathbf{Q}_k$$

$$\mathbf{K}_k = \mathbf{M}_k \mathbf{H}^T (\mathbf{H} \mathbf{M}_k \mathbf{H}^T + \mathbf{R}_k)^{-1}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{M}_k$$

where $\mathbf{P}_k$ is a covariance matrix representing errors in the state estimates after an update and $\mathbf{M}_k$ is the covariance matrix representing errors in the state estimates before an update. As was just mentioned, because Φ_k and $\mathbf{H}$ can be considered linear functions of time, the Kalman gains can also be computed offline, as is

possible with a linear Kalman filter. The discrete process-noise matrix $\mathbf{Q}_k$ can still be found from the continuous process-noise matrix, according to

$$\mathbf{Q}_k = \int_0^{T_s} \Phi(\tau) \mathbf{Q} \Phi^T(\tau) d\tau$$

Finally, the equation for the linearized Kalman filter is given by

$$\Delta \hat{\mathbf{x}}_k = \Phi \Delta \hat{\mathbf{x}}_{k-1} + \mathbf{K}_k (\Delta \mathbf{z}_k - \mathbf{H} \Phi \Delta \hat{\mathbf{x}}_{k-1})$$

We have just seen that the linearized Kalman filter gives estimates of the error states. Therefore, in order to find estimates of the actual states we must combine nominal trajectory information with the error state estimates or

$$\hat{\mathbf{x}}_k = \Delta \hat{\mathbf{x}}_k + \mathbf{x}_{\text{NOM}}$$

where $\mathbf{x}_{\text{NOM}}$ are the states obtained from the nominal trajectory. The best way to understand the equations for the linearized Kalman filter is by working an example.

Falling Object Revisited

Let us again consider the one-dimensional example of an object falling on a tracking radar, as was shown in Fig. 8.1 but now redrawn for convenience in Fig. 13.1. In this problem we are assuming that the drag is unknown, which means we have to estimate the ballistic coefficient in order to get a good estimate of the position and velocity of the object. The ballistic coefficient of the object in this example is 500 lb/ft². As was the case in Chapter 8, the object was initially at 200,000 ft above the radar and had a velocity of 6000 ft/s toward the radar that

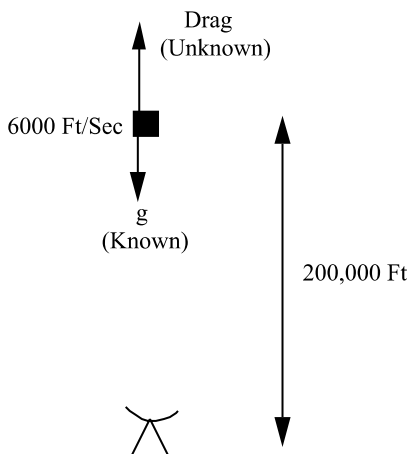


Fig. 13.1 Radar tracking object with unknown drag.

is located on the surface of the Earth. The radar measures the range from the radar to the object (i.e., altitude of the object) with a 25-ft standard deviation measurement accuracy. The radar takes measurements 10 times a second until ground impact. Let us assume that we have nominal trajectory information for a 6000-ft/s falling object with a ballistic coefficient of 500 lb/ft² starting at an altitude of 200,000 ft. We would like to build a linearized Kalman filter to estimate the altitude, velocity, and ballistic coefficient of the object based on both the nominal trajectory information and the radar measurements. *The following is an academic exercise because if this type of information were really available it might not be necessary to build the estimator.*

As was the case in Chapter 8, if x is the distance from the radar to the object (i.e., altitude), then the acceleration in the altitude direction acting on the object consists of gravity and drag or

$$\ddot{x} = \text{Drag} - g = \frac{Q_p g}{\beta} - g$$

where β is the ballistic coefficient of the object. The ballistic coefficient, which is considered to be a constant in this problem, is a term describing the amount of drag acting on the object. The dynamic pressure Q_p in the preceding equation is given by

$$Q_p = 0.5\rho\dot{x}^2$$

where ρ is the air density and $\dot{x}$ is the velocity of the object in the altitude direction. As was the case in Chapter 8, we can still assume that the air density is an exponential function of altitude² given by

$$\rho = 0.0034e^{-x/22,000}$$

Therefore, the second-order differential equation describing the acceleration on the object can be expressed as

$$\ddot{x} = \frac{Q_p g}{\beta} - g = \frac{0.5g\rho\dot{x}^2}{\beta} - g = \frac{0.0034ge^{-x/22,000}\dot{x}^2}{2\beta} - g$$

Before we proceed any further, it is of interest to see how accurate the nominal trajectory is for our problem. Listing 13.1 presents a simulation in which both the actual and nominal equations of motion of the object are both integrated. We can see from Listing 13.1 that the initial conditions for the nominal and actual trajectories of the object are identical. The actual ballistic coefficient is 500 lb/ft², and the nominal ballistic coefficient is also 500 lb/ft². We will keep the actual ballistic coefficient fixed but vary the nominal ballistic coefficient from 500 to 1100 lb/ft² (i.e., a change of more than 100%) in steps of 300 lb/ft². Every 0.1 s we print out the difference between the actual and nominal altitudes, actual and nominal velocities, and actual and nominal ballistic coefficients. In addition, we also print out the actual and nominal altitudes and velocities of the object.

Listing 13.1 Simulation for actual and nominal trajectories of falling object

```

IMPLICIT REAL*8 (A-H)
IMPLICIT REAL*8 (O-Z)
X=200000.
XD=-6000.
BETA=500.
XNOM=200000.
XDNOM=-6000.
BETANOM=800.
OPEN(1,STATUS='UNKNOWN',FILE='DATFIL')
TS=.1
TS=0.
S=0.
H=.001
WHILE(X>=0.)
    XOLD=X
    XDOLD=XD
    XNOMOLD=XNOM
    XDNOMOLD=XDNOM
    XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
    XDDNOM=.0034*32.2*XDNOM*XDNOM*EXP(-XNOM/22000.)/
1      (2.*BETANOM)-32.2
    X=X+H*XD
    XD=XD+H*XDD
    XNOM=XNOM+H*XDNOM
    XDNOM=XDNOM+H*XDDNOM
    T=T+H
    XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
    XDDNOM=.0034*32.2*XDNOM*XDNOM*EXP(-XNOM/22000.)/
1      (2.*BETANOM)-32.2
    X=.5*(XOLD+X+H*XD)
    XD=.5*(XDOLD+XD+H*XDD)
    XNOM=.5*(XNOMOLD+XNOM+H*XDNOM)
    XDNOM=.5*(XDNOMOLD+XDNOM+H*XDDNOM)
    S=S+H
    IF(S>=(TS-.00001))THEN
        S=0.
        DELXH=X-XNOM
        DELXDH=XD-XDNOM
        DELBETAH=BETA-BETANOM
        XH=XNOM+DELXH
        XDH=XDNOM+DELXDH
        BETAH=BETANOM+DELBETAH
        WRITE(9,*)T,DELXH,DELXDH,DELBETAH,X,XH,
1      XD,XDH
        WRITE(1,*)T,DELXH,DELXDH,DELBETAH,X,XH,
1      XD,XDH
    ENDIF
END DO
PAUSE
CLOSE(1)
END

```

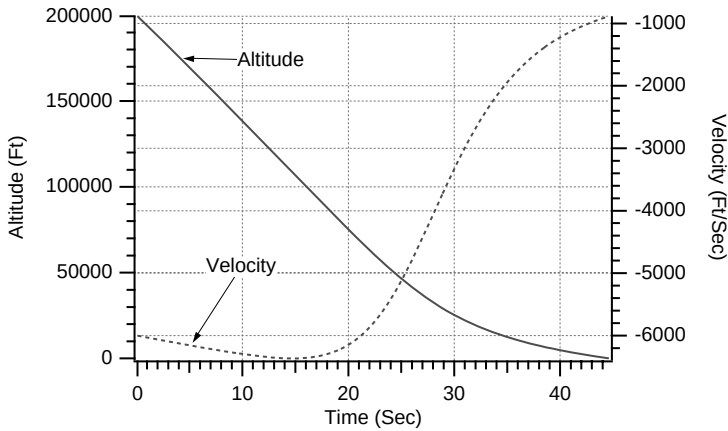


Fig. 13.2 Object slows up significantly as it races toward the ground.

The nominal case of Listing 13.1 was run in which the actual and nominal trajectories were identical. Figure 13.2 presents the altitude and velocity profiles for the falling object. We can see that it takes nearly 45 s for the falling object to reach the ground from its initial altitude of 200,000 ft. The velocity of the object decreases from 6000 ft/s at 200,000 ft altitude to approximately 1000 ft/s near ground impact.

Figures 13.3 and 13.4 show how the differences between the actual and nominal altitude and velocity occur when the nominal value of the ballistic coefficient differs from the actual value of the ballistic coefficient. We can see from Fig. 13.3 that a 100 lb/ft² difference in the ballistic coefficient (i.e., $\beta_{ACT} = 500$ lb/ft² and $\beta_{NOM} = 600$ lb/ft² or a 20% difference) can yield a 4000-ft altitude error, whereas we can see from Fig. 13.4 that the same error in ballistic coefficient yields a 50-ft/s velocity error by the time the object hits the

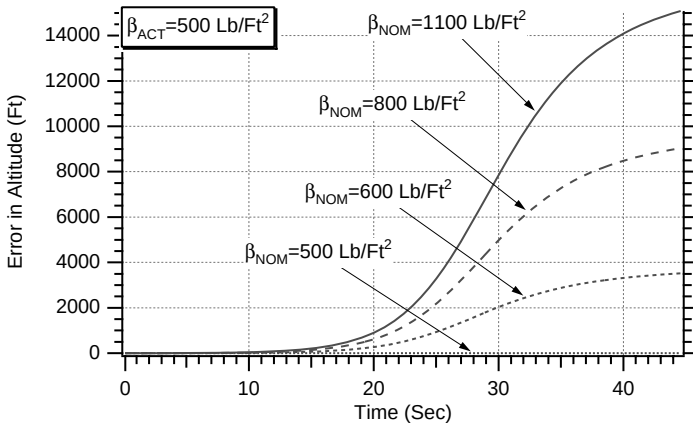


Fig. 13.3 Differences between actual and nominal ballistic coefficient can yield significant differences between actual and nominal altitude of object.

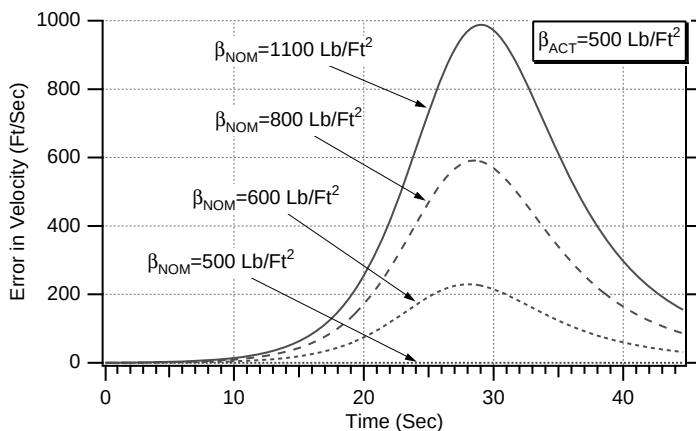


Fig. 13.4 Differences between actual and nominal ballistic coefficient can yield significant differences between actual and nominal velocity of object.

ground. A 300-lb/ft² difference in the ballistic coefficient (i.e., $\beta_{\text{ACT}} = 500 \text{ lb/ft}^2$ and $\beta_{\text{NOM}} = 800 \text{ lb/ft}^2$ or a 60% difference) can yield double the altitude and velocity errors (i.e., 8000-ft altitude error and 100-ft/s velocity error). We can also see from Fig. 13.3 that a 600-lb/ft² difference in the ballistic coefficient (i.e., $\beta_{\text{ACT}} = 500 \text{ lb/ft}^2$ and $\beta_{\text{NOM}} = 1100 \text{ lb/ft}^2$ or a 120% difference) can yield a 14,000-ft altitude error, and from Fig. 13.4 we can see that the same ballistic coefficient error yields a 200-ft/s velocity error by the time the object hits the ground.

Developing a Linearized Kalman Filter

In Chapter 8 the states of the extended Kalman filter were altitude, velocity, and ballistic coefficient. Therefore, it makes sense that the error states for the linearized Kalman filter used to track the falling object should be

$$\Delta \mathbf{x} = \begin{bmatrix} \Delta x \\ \Delta \dot{x} \\ \Delta \beta \end{bmatrix} = \begin{bmatrix} x - x_{\text{NOM}} \\ \dot{x} - \dot{x}_{\text{NOM}} \\ \beta - \beta_{\text{NOM}} \end{bmatrix}$$

Recall that the acceleration acting on the object is a nonlinear function of the states and was shown to be

$$\ddot{x} = \frac{0.0034 g e^{-x/22,000} \dot{x}^2}{2\beta} - g$$

In addition, if we believe that the ballistic coefficient is constant, that means its derivative must be zero or

$$\dot{\beta} = 0$$

For safety we can add process noise u_s to this equation to account for the fact that the ballistic coefficient may be varying. In that case the new equation for the derivative of the ballistic coefficient becomes

$$\dot{\beta} = u_s$$

Therefore, our model of the real world is given by

$$\dot{\mathbf{x}} = f(\mathbf{x}) + \mathbf{w}$$

and we want the error equations to be of the form

$$\Delta \dot{\mathbf{x}} = \left. \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{NOM}}} \Delta \mathbf{x} + \mathbf{w}$$

Because we have chosen the appropriate states for this tracking problem, substitution yields our error equation model of the real world. As was already mentioned, in this equation the partial derivatives are evaluated along the nominal trajectory

$$\begin{bmatrix} \Delta \dot{x} \\ \Delta \ddot{x} \\ \Delta \dot{\beta} \end{bmatrix} = \left[\begin{array}{ccc} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial \dot{x}} & \frac{\partial \dot{x}}{\partial \beta} \\ \frac{\partial \ddot{x}}{\partial x} & \frac{\partial \ddot{x}}{\partial \dot{x}} & \frac{\partial \ddot{x}}{\partial \beta} \\ \frac{\partial \dot{\beta}}{\partial x} & \frac{\partial \dot{\beta}}{\partial \dot{x}} & \frac{\partial \dot{\beta}}{\partial \beta} \end{array} \right]_{\mathbf{x}=\mathbf{x}_{\text{NOM}}} \begin{bmatrix} \Delta x \\ \Delta \dot{x} \\ \Delta \beta \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ u_s \end{bmatrix}$$

We can see from the preceding linearized state-space differential equation that the systems dynamic matrix is the matrix of partial derivatives evaluated along the nominal trajectory or

$$\mathbf{F} = \left[\begin{array}{ccc} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial \dot{x}} & \frac{\partial \dot{x}}{\partial \beta} \\ \frac{\partial \ddot{x}}{\partial x} & \frac{\partial \ddot{x}}{\partial \dot{x}} & \frac{\partial \ddot{x}}{\partial \beta} \\ \frac{\partial \dot{\beta}}{\partial x} & \frac{\partial \dot{\beta}}{\partial \dot{x}} & \frac{\partial \dot{\beta}}{\partial \beta} \end{array} \right]_{\mathbf{x}=\mathbf{x}_{\text{NOM}}}$$

We can also see from the linearized state-space equation that the continuous process-noise matrix is given by

$$\mathbf{Q} = E(\mathbf{w}\mathbf{w}^T)$$

Therefore, we can say that for this example the continuous process-noise matrix is

$$\mathbf{Q}(t) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \Phi_s \end{bmatrix}$$

where Φ_s is the spectral density of the white noise source assumed to be on the derivative of the error equation for the ballistic coefficient. To evaluate the systems dynamics matrix, we must take the necessary partial derivatives as indicated by the systems dynamics matrix $\mathbf{F}$. The first row of partial derivatives yields

$$\frac{\partial \dot{x}}{\partial x} = 0$$

$$\frac{\partial \dot{x}}{\partial \dot{x}} = 1$$

$$\frac{\partial \dot{x}}{\partial \beta} = 0$$

whereas the second row of partial derivatives yields

$$\frac{\partial \ddot{x}}{\partial x} = \frac{-0.0034e^{-x/22,000} \dot{x}^2 g}{2\beta(22,000)} = \frac{-\rho g \dot{x}^2}{44,000\beta}$$

$$\frac{\partial \ddot{x}}{\partial \dot{x}} = \frac{2*0.0034e^{-x/22,000} \dot{x} g}{2\beta} = \frac{\rho g \dot{x}}{\beta}$$

$$\frac{\partial \ddot{x}}{\partial \beta} = \frac{-0.0034e^{-x/22,000} \dot{x}^2 g}{2\beta^2} = \frac{-\rho g \dot{x}^2}{2\beta^2}$$

Finally, the last row of partial derivatives yields

$$\frac{\partial \dot{\beta}}{\partial x} = 0$$

$$\frac{\partial \dot{\beta}}{\partial \dot{x}} = 0$$

$$\frac{\partial \dot{\beta}}{\partial \beta} = 0$$

Therefore, the systems dynamics matrix turns out to be

$$\mathbf{F} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{-\rho_{\text{NOM}} g \dot{x}_{\text{NOM}}^2}{44,000 \beta_{\text{NOM}}} & \frac{\rho_{\text{NOM}} g \dot{x}_{\text{NOM}}}{\beta_{\text{NOM}}} & \frac{-\rho_{\text{NOM}} g \dot{x}_{\text{NOM}}^2}{2 \beta_{\text{NOM}}^2} \\ 0 & 0 & 0 \end{bmatrix}$$

where gravity g is assumed to be known perfectly and the other quantities are obtained from the nominal trajectory. The nominal air density in the preceding expression is given by

$$\rho_{\text{NOM}} = 0.0034 e^{-x_{\text{NOM}}/22,000}$$

The fundamental matrix can be found by a two-term Taylor-series approximation, yielding

$$\Phi(t) \approx \mathbf{I} + \mathbf{F}t$$

From the systems dynamics matrix we see that it make sense to define

$$\begin{aligned} f_{21} &= \frac{-\rho_{\text{NOM}} g \dot{x}_{\text{NOM}}^2}{44,000 \beta_{\text{NOM}}} \\ f_{22} &= \frac{\rho_{\text{NOM}} \dot{x}_{\text{NOM}} g}{\beta_{\text{NOM}}} \\ f_{23} &= \frac{-\rho_{\text{NOM}} g \dot{x}_{\text{NOM}}^2}{2 \beta_{\text{NOM}}^2} \end{aligned}$$

Now the systems dynamics matrix can be written more compactly as

$$\mathbf{F}(t) = \begin{bmatrix} 0 & 1 & 0 \\ f_{21} & f_{22} & f_{23} \\ 0 & 0 & 0 \end{bmatrix}$$

If we only use the first two terms of the Taylor-series expansion for the fundamental matrix, we obtain

$$\Phi(t) \approx \mathbf{I} + \mathbf{F}t = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ f_{21} & f_{22} & f_{23} \\ 0 & 0 & 0 \end{bmatrix} t = \begin{bmatrix} 1 & t & 0 \\ f_{21}t & 1 + f_{22}t & f_{23}t \\ 0 & 0 & 1 \end{bmatrix}$$

or more simply

$$\Phi(t) \approx \begin{bmatrix} 1 & t & 0 \\ f_{21}t & 1 + f_{22}t & f_{23}t \\ 0 & 0 & 1 \end{bmatrix}$$

Therefore, the discrete fundamental matrix can be found by substituting T_s for t or

$$\Phi_k \approx \begin{bmatrix} 1 & T_s & 0 \\ f_{21}T_s & 1 + f_{22}T_s & f_{23}T_s \\ 0 & 0 & 1 \end{bmatrix}$$

In this example the error altitude measurement is a linear function of the states and is given by

$$\Delta x^* = [1 \quad 0 \quad 0] \begin{bmatrix} \Delta x \\ \Delta \dot{x} \\ \Delta \beta \end{bmatrix} + v$$

where v is the measurement noise. We can see from the preceding equation that the measurement noise matrix can be written by inspection and is given by

$$\mathbf{H} = [1 \quad 0 \quad 0]$$

Because the discrete measurement noise matrix is given by

$$\mathbf{R}_k = E(\mathbf{v}_k \mathbf{v}_k^T)$$

we can say that for this problem the discrete measurement noise matrix turns out to be a scalar and is given by

$$\mathbf{R}_k = \sigma_v^2$$

where σ_v^2 is the variance of the altitude noise. We have already stated that the continuous process-noise matrix is given by

$$\mathbf{Q} = E(\mathbf{w} \mathbf{w}^T)$$

Because the process noise is on the derivative of the ballistic coefficient, the continuous process-noise matrix for this example is given by

$$\mathbf{Q}(t) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \Phi_s \end{bmatrix}$$

where Φ_s is the spectral density of the white noise source assumed to be on the derivative of the third state. The discrete process-noise matrix can be derived from the continuous process-noise matrix according to

$$\mathbf{Q}_k = \int_0^{T_s} \Phi(\tau) \mathbf{Q} \Phi^T(\tau) d\tau$$

Therefore, substitution yields

$$\mathbf{Q}_k = \Phi_s \int_0^{T_s} \begin{bmatrix} 1 & \tau & 0 \\ f_{21}\tau & 1 + f_{22}\tau & f_{23}\tau \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & f_{21}\tau & 0 \\ \tau & 1 + f_{22}\tau & 0 \\ 0 & f_{23}\tau & 0 \end{bmatrix} d\tau$$

After some algebra we get

$$\mathbf{Q}_k = \Phi_s \int_0^{T_s} \begin{bmatrix} 0 & 0 & 0 \\ 0 & f_{23}^2 \tau^2 & f_{23} \tau \\ 0 & f_{23} \tau & 1 \end{bmatrix} d\tau$$

Finally, after integration we obtain the final expression for the discrete process-noise matrix as

$$\mathbf{Q}_k = \Phi_s \begin{bmatrix} 0 & 0 & 0 \\ 0 & f_{23}^2 \frac{T_s^3}{3} & f_{23} \frac{T_s^2}{2} \\ 0 & f_{23} \frac{T_s^2}{2} & T_s \end{bmatrix}$$

We now have defined all of the matrices required to solve the Riccati equations. The equation for the linearized Kalman filter is given by

$$\begin{bmatrix} \Delta \hat{\mathbf{x}}_k \\ \Delta \hat{\mathbf{x}}_k \\ \Delta \hat{\beta}_k \end{bmatrix} = \Phi \begin{bmatrix} \Delta \hat{\mathbf{x}}_{k-1} \\ \Delta \hat{\mathbf{x}}_{k-1} \\ \Delta \hat{\beta}_{k-1} \end{bmatrix} + \begin{bmatrix} K_{1k} \\ K_{2k} \\ K_{3k} \end{bmatrix} \left(\Delta \mathbf{x}_k^* - \mathbf{H} \Phi \begin{bmatrix} \Delta \hat{\mathbf{x}}_{k-1} \\ \Delta \hat{\mathbf{x}}_{k-1} \\ \Delta \hat{\beta}_{k-1} \end{bmatrix} \right)$$

where $\Delta \mathbf{x}_k^*$ is the noisy measurement of the difference between the actual and nominal altitudes, and the measurement and fundamental matrices have been defined already.

The preceding equations for the Kalman filter and Riccati equations were programmed and are shown in Listing 13.2, along with a simulation of the real world and a simulation of the nominal trajectory. From Listing 13.2 we can see that initially the nominal and actual ballistic coefficients are identical and are set to 500 lb/ft². Therefore, initially we have a perfect nominal trajectory because it is identical to the actual nominal trajectory. We can see that initially there is

Listing 13.2 Simulation of linearized Kalman filter and falling object

C THE FIRST THREE STATEMENTS INVOKE THE ABSOFT RANDOM
NUMBER GENERATOR ON THE MACINTOSH

```

GLOBAL DEFINE
    INCLUDE 'quickdraw.inc'
END
IMPLICIT REAL*8 (A-H)
IMPLICIT REAL*8 (O-Z)
REAL*8 PHI(3,3),P(3,3),PHIP(3,3),PHIPPHIT(3,3),K(3,1)
REAL*8 Q(3,3),HMAT(1,3),HM(1,3),MHT(3,1),XH(3,1)
REAL*8 PHIT(3,3)
REAL*8 HMHT(1,1),HT(3,1),KH(3,3),IDN(3,3),IKH(3,3)
REAL*8 MEAS(1,1),PHIXH(3,1),HPHIXH(1,1),RES(1,1),KRES(3,1)
INTEGER ORDER
X=200000.
XD=-6000.
BETA=500.
XNOM=200000.
XDNOM=-6000.
BETANOM=500.
OPEN(1,STATUS='UNKNOWN',FILE='DATFIL')
ORDER=3
TS=.1
TF=30.
Q33=300.*300./TF
T=0.
S=0.
H=.001
SIGNOISE=25.
XH(1,1)=X-XNOM
XH(2,1)=XD-XDNOM
XH(3,1)=-300.
DO 1000 I=1,ORDER
DO 1000 J=1,ORDER
    PHI(I,J)=0.
    P(I,J)=0.
    Q(I,J)=0.
    IDN(I,J)=0.
1000 CONTINUE
IDN(1,1)=1.
IDN(2,2)=1.
IDN(3,3)=1.
P(1,1)=SIGNOISE*SIGNOISE
P(2,2)=20000.
P(3,3)=300.**2
DO 1100 I=1,ORDER
    HMAT(1,I)=0.
    HT(I,1)=0.
1100 CONTINUE
HMAT(1,1)=1.
HT(1,1)=1.

```

(continued)

Listing 13.2 (Continued)

```

WHILE(X>=0.)
  XOLD=X
  XDOLD=XD
  XNOMOLD=XNOM
  XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
  XDDNOM=.0034*32.2*XDNOM*XDNOM*EXP(-XNOM/22000.)/
1    (2.*BETANOM)-32.2
  X=X+H*XDD
  XD=XD+H*XD
  XNOM=XNOM+H*XDNOM
  XDNOM=XDNOM+H*XDDNOM
  T=T+H
  XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
  XDDNOM=.0034*32.2*XDNOM*XDNOM*EXP(-XNOM/22000.)/
1    (2.*BETANOM)-32.2
  X=.5*(XOLD+X+H*XD)
  XD=.5*(XDOLD+XD+H*XDD)
  XNOM=.5*(XNOMOLD+XNOM+H*XDNOM)
  XDNOM=.5*(XDNOMOLD+XDNOM+H*XDDNOM)
  S=S+H
  IF(S>=(TS-.00001))THEN
    S=0.
    RHONOM=.0034*EXP(-XNOM/22000.)
    F21=-32.2*RHONOM*XDNOM*XDNOM/(2.*22000.
1      *BETANOM)
    F22=RHONOM*32.2*XDNOM/BETANOM
    F23=-RHONOM*32.2*XDNOM*XDNOM/(2.*BETANOM*
1      BETANOM)
    PHI(1,1)=1.
    PHI(1,2)=TS
    PHI(2,1)=F21*TS
    PHI(2,2)=1.+F22*TS
    PHI(2,3)=F23*TS
    PHI(3,3)=1.
    Q(2,2)=F23*F23*Q33*TS*TS*TS/3.
    Q(2,3)=F23*Q33*TS*TS/2.
    Q(3,2)=Q(2,3)
    Q(3,3)=Q33*TS
    CALL MATTRN(PHI,ORDER,ORDER,PHIT)
    CALL MATMUL(PHI,ORDER,ORDER,P,ORDER,ORDER,PHIP)
    CALL MATMUL(PHIP,ORDER,ORDER,PHIT,ORDER,ORDER.
1      PHIPPHIT)
    CALL MATADD(PHIPPHIT,ORDER,ORDER,Q,M)
    CALL MATMUL(HMAT,1,ORDER,M,ORDER,ORDER,HM)
    CALL MATMUL(HM,1,ORDER,HT,ORDER,1,HMHT)
    HMHTR=HMHT(1,1)+SIGNOISE*SIGNOISE
    HMHTRINV=1./HMHTR
    CALL MATMUL(M,ORDER,ORDER,HT,ORDER,1,MHT)
    DO 150 I=1,ORDER
      K(I,1)=MHT(I,1)*HMHTRINV

```

(continued)

Listing 13.2 (Continued)

```

150      CONTINUE
        CALL MATMUL(K,ORDER,1,HMAT,1,ORDER,KH)
        CALL MATSUB(IDN,ORDER,ORDER,KH,IKH)
        CALL MATMUL(IKH,ORDER,ORDER,M,ORDER,ORDER,P)
        CALL GAUSS(XNOISE,SIGNOISE)
        DELX=X-XNOM
        MEAS(1,1)=DELX+XNOISE
        CALL MATMUL(PHI,ORDER,ORDER,XH,ORDER,1,PHIXH)
        CALL MATMUL(HMAT,1,ORDER,PHIXH,ORDER,1,HPHIXH)
        CALL MATSUB(MEAS,1,1,HPHIXH,RES)
        CALL MATMUL(K,ORDER,1,RES,1,1,KRES)
        CALL MATADD(PHIXH,ORDER,1,KRES,XH)
        DELXH=XH(1,1)
        DELXDH=XH(2,1)
        DELBETAH=XH(3,1)
        XHH=XNOM+DELXH
        XDH=XDNOM+DELXDH
        BETAH=BETANOM+DELBETAH
        ERRX=X-XHH
        SP11=SQRT(P(1,1))
            ERRXD=XD-XDH
            SP22=SQRT(P(2,2))
            ERRBETA=BETA-BETAH
            SP33=SQRT(P(3,3))
            WRITE(9,*)T,BETA,BETAH,ERRBETA,SP33,-SP33
            WRITE(1,*)T,BETA,BETAH,ERRBETA,SP33,-SP33
        ENDIF
    END DO
    PAUSE
    CLOSE(1)
    END

C SUBROUTINE GAUSS IS SHOWN IN LISTING 1.8
C SUBROUTINE MATTRN IS SHOWN IN LISTING 1.3
C SUBROUTINE MATMUL IS SHOWN IN LISTING 1.4
C SUBROUTINE MATADD IS SHOWN IN LISTING 1.1
C SUBROUTINE MATSUB IS SHOWN IN LISTING 1.2

```

process noise and that the third diagonal element of the initial covariance matrix has been set to 300^2 to reflect what we believe the uncertainty to be in our first guess of the estimated ballistic coefficient. This, of course, is simulation fiction because we really know the initial ballistic coefficient perfectly by virtue of the fact that the ballistic coefficient of the nominal trajectory corresponds to truth. We have initialized the error states estimates of the filter to zero, except for the ballistic coefficient error state, which we have set to -300 lb/ft^2 . The standard deviation of the measurement noise on our error altitude measurement is 25 ft.

The nominal case was run, and the ballistic coefficient estimate profile is displayed in Fig. 13.5. We can see that the linearized Kalman filter is able to

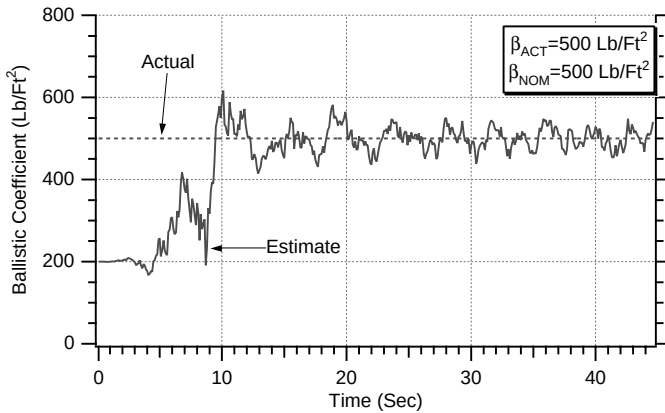


Fig. 13.5 Linearized Kalman filter is able to accurately estimate the ballistic coefficient after only 10 s.

accurately track the ballistic coefficient after only 10 s. The estimates could be made less noisy by simply reducing the value of the process noise. Figure 13.6 shows that the linearized Kalman filter is working correctly because the singly run simulation errors in the estimates of ballistic coefficient lie within the theoretical bounds.

Another case was run with Listing 13.2 in which the nominal value of the ballistic coefficient was increased by 20% to 600 lb/ft². This means that the nominal trajectory, upon which the linearized Kalman filter is based, will be in slight error. All other parameters in Listing 13.2 were left alone. Figure 13.7 shows that the ballistic coefficient estimate is still excellent, whereas Fig. 13.8

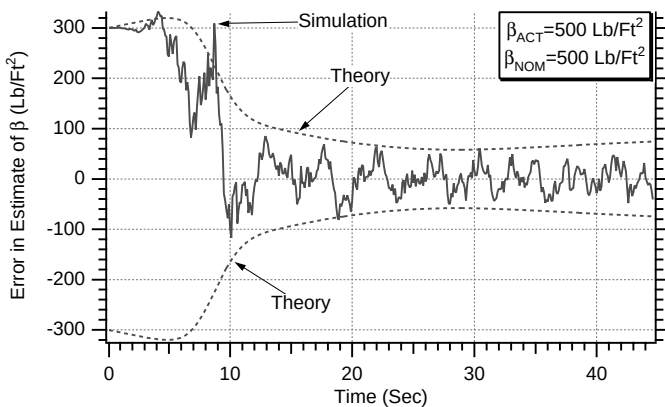


Fig. 13.6 Linearized Kalman filter appears to be working correctly because the single-run errors in the estimates of the ballistic coefficient lie within the theoretical bounds.

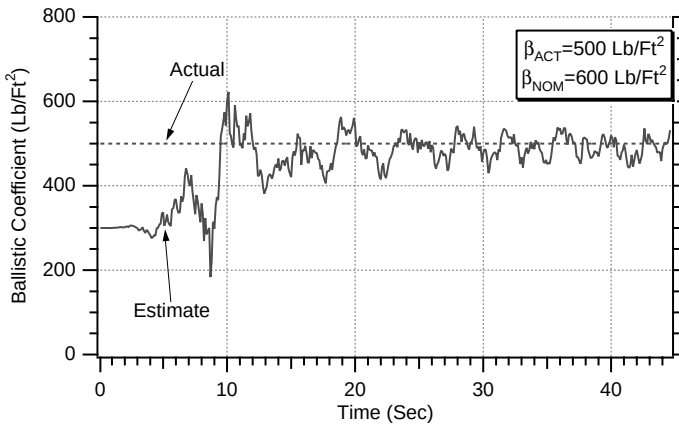


Fig. 13.7 Linearized Kalman filter is able to estimate accurately the ballistic coefficient when there is a slight error in the nominal trajectory.

indicates that the errors in the estimate of the ballistic coefficient are still within the theoretical bounds. Again, this also holds true after many simulation trials. We still can say that the linearized Kalman filter is performing according to theory.

Another case was run with Listing 13.2 in which the nominal value of the ballistic coefficient was increased by 60% to 800 lb/ft². This means that the nominal trajectory, upon which the linearized Kalman filter is based, will be in greater error than the preceding case. Figure 13.9 shows that the ballistic coefficient estimate is always less than the actual ballistic coefficient. This appears to be true even after many repeated simulation trials. Figure 13.10 indicates that now the errors in the estimate of the ballistic coefficient are no

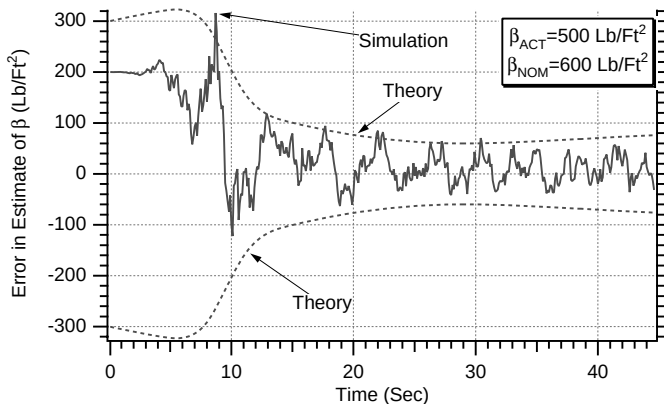


Fig. 13.8 Error in estimate of the ballistic coefficient is between theoretical error bounds when there is a slight error in the nominal trajectory.

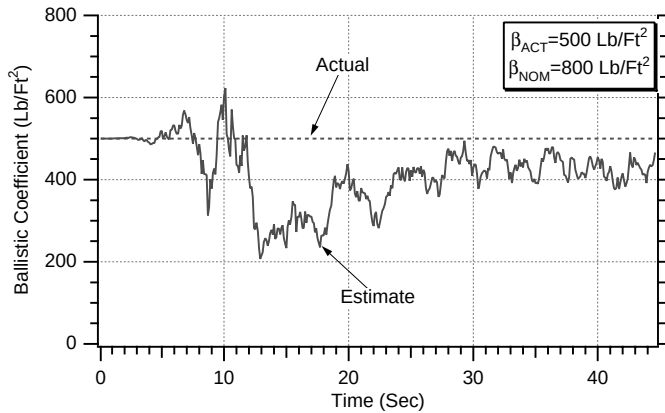


Fig. 13.9 Linearized Kalman filter underestimates the ballistic coefficient when there is 60% error in the nominal ballistic coefficient.

longer within the theoretical bounds but slightly on top of the upper bound. Again, this also holds true after many simulation trials. In this case we cannot say that the linearized Kalman filter is performing according to theory.

Finally, another case was run with Listing 13.2 in which the nominal value of the ballistic coefficient was increased 120% to 1100 lb/ft². This means that the nominal trajectory, upon which the linearized Kalman filter is based, was in even greater error than in the preceding case. Figure 13.11 shows that the ballistic coefficient estimate is not only always less than the actual ballistic coefficient but also has a significant steady-state error. This also appears to be true after many repeated simulation trials. Figure 13.12 indicates that now the errors in the

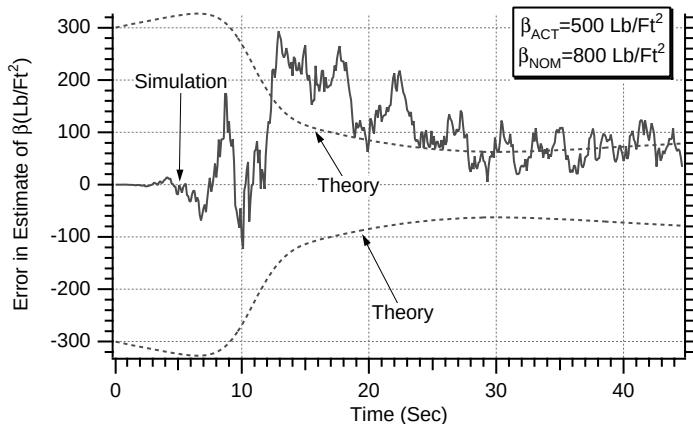


Fig. 13.10 Error in estimate of the ballistic coefficient is not between theoretical error bounds when there is a 60% error in the nominal ballistic coefficient.

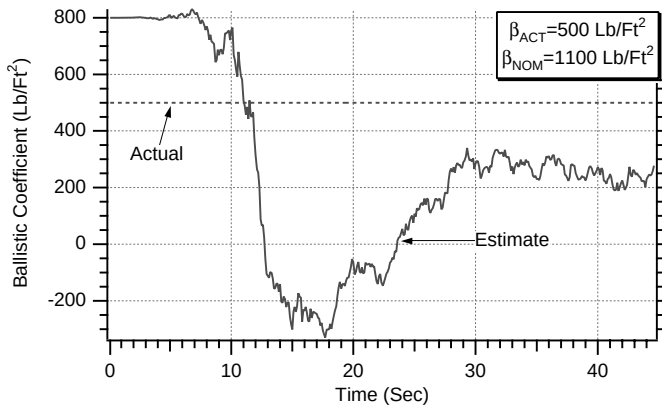


Fig. 13.11 Significant steady-state error in the linearized Kalman filter's estimate of the ballistic coefficient exists when there is a 120% error in the nominal ballistic coefficient.

estimate of the ballistic coefficient are completely outside the theoretical error bounds. This also holds true after many simulation trials. In this case we can definitely say that the linearized Kalman filter is not performing according to theory.

Experiments also were conducted in which the nominal ballistic coefficient was decreased from 500 to 100 lb/ft² in steps of 100 lb/ft². The linearized Kalman filter gave excellent estimates when the nominal ballistic coefficient was reduced 20% to 400 lb/ft². The estimates deteriorated somewhat when the nominal ballistic coefficient was reduced 40% to 300 lb/ft². In this case, the errors in the estimates were on the upper theoretical bound. However, when the nominal

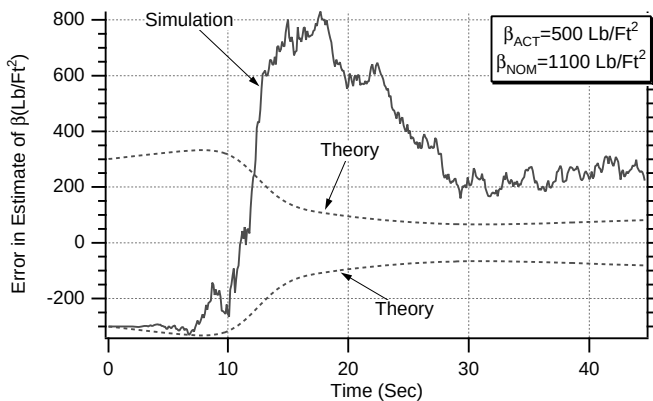


Fig. 13.12 When there is a 120% error in the nominal ballistic coefficient, the errors in the estimates from the linearized Kalman filter are completely outside the theoretical error bounds.

ballistic coefficient was reduced 60% to 200 lb/ft², the errors in the estimates were always outside the theoretical error bounds.

Thus, for this example, we can say that the linearized Kalman filter only performed well when the nominal trajectory was close to the actual trajectory (i.e., nominal ballistic coefficient differed from actual ballistic coefficient by no more than 20%). When the nominal trajectory was significantly different from the actual trajectory, the linearized Kalman filter's estimates diverged from the actual values of the ballistic coefficient.

Cannon-Launched Projectile Revisited

Let us revisit the cannon-launched projectile problem of Chapter 9. In this simple example a radar tracks a cannon-launched projectile traveling in a two-dimensional, drag-free environment, as shown in Fig. 13.13. After the projectile is launched at an initial velocity, only gravity acts on the projectile. The tracking radar that is located at coordinates x_R, y_R of Fig. 9.1 measures the range r and angle θ from the radar to the projectile.

In this example the projectile is launched at an initial velocity of 3000 ft/s at a 45-deg angle with respect to the horizontal axis in a zero-drag environment (i.e., atmosphere is neglected). The radar is located on the ground and is 100,000 ft downrange from where the projectile is initially launched. The radar is considered to have an angular measurement accuracy of 0.01 rad and a range measurement accuracy of 100 ft.

Before we can proceed with the development of a filter for this application, it is important to get a numerical and analytical feel for all aspects of the problem. Because only gravity acts on the projectile, we can say that in the downrange or x direction there is no acceleration, whereas in the altitude or y direction there is the acceleration of gravity g or

$$\begin{aligned}\ddot{x}_T &= 0 \\ \ddot{y}_T &= -g\end{aligned}$$

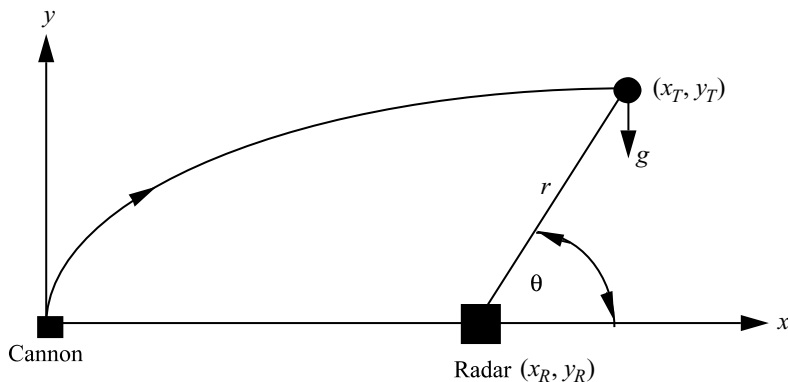


Fig. 13.13 Radar tracking cannon-launched projectile.

where g is the acceleration of gravity and has the value of 32.2 ft/s^2 in the English system of units. Using trigonometry, the angle and range from the radar to the projectile can be obtained by inspection of Fig. 13.13 to be

$$\theta = \tan^{-1} \left(\frac{y_T - y_R}{x_T - x_R} \right)$$

$$r = \sqrt{(x_T - x_R)^2 + (y_T - y_R)^2}$$

Before we proceed any further it is of interest to see how accurate the nominal trajectory is for our problem. Listing 13.3 presents a simulation in which the actual and nominal equations of motion of the object are both integrated. We can see from the listing that the initial conditions for the nominal and actual trajectories of the object are identical. The projectile velocity at launch is 3000 ft/s , whereas the nominal projectile velocity is also 3000 ft/s . We will keep the actual initial projectile velocity fixed but vary the nominal projectile velocity from 3050 to 3200 ft/s . Every second we print out the difference between the actual and nominal downranges, actual and nominal radar angles, and actual and nominal radar ranges.

Figures 13.14–13.16 show how the differences between the actual and nominal downranges, radar angles, and radar ranges vary when the nominal value of the initial projectile velocity differs from the actual initial value of the projectile velocity. We can see from Fig. 13.14 that a 50-ft/s difference or 1.7% increase in the initial projectile velocity can yield a 3000-ft downrange error. We can also see that a 100-ft/s difference or 3.3% increase in the initial projectile velocity can yield a 9000-ft downrange error. Finally, we can see that a 200-ft/s difference or 6.7% increase in the initial projectile velocity can yield an $18,000\text{-ft}$ downrange error. Thus, we can see that small initial velocity errors can yield substantial errors in the downrange component of the nominal trajectory. Figure 13.15 indicates that a 50-ft/s difference or 1.7% increase in the initial projectile velocity can yield a 1-deg radar angle error. The figure also indicates that a 100-ft/s difference or 3.3% increase in the initial projectile velocity can yield a 3-deg radar angle error, whereas a 200-ft/s difference or 6.7% increase in the initial projectile velocity can yield a 5-deg radar angle error. Similarly, Fig. 13.16 indicates that a 50-ft/s difference or 1.7% increase in the initial projectile velocity can yield a 4000-ft radar range error, whereas a 100-ft/s difference or 3.3% increase in the initial projectile velocity can yield an 8000-ft radar range error. Figure 13.16 also shows that a 200-ft/s difference or 6.7% increase in the initial projectile velocity can yield a $20,000\text{-ft}$ radar range error.

Linearized Cartesian Kalman Filter

In Chapter 9 the states of the extended Kalman filter were downrange, downrange velocity, altitude, and altitude velocity. Therefore, it makes sense

Listing 13.3 Simulation for actual and nominal projectile trajectories

```

IMPLICIT REAL*8(A-H,O-Z)
TS=1.
VT=3000.
VTERR=0.
GAMDEG=45.
GAMDEGERR=0.
VTNOM=VT+VTERR
GAMDEGNOM=GAMDEG+GAMDEGERR
G=32.2
XT=0.
YT=0.
XTD=VT*COS(GAMDEG/57.3)
YTD=VT*SIN(GAMDEG/57.3)
XTNOM=XT
YTNOM=YT
XTDNOM=VTNOM*COS(GAMDEGNOM/57.3)
YTDNOM=VTNOM*SIN(GAMDEGNOM/57.3)
XR=100000.
YR=0.
OPEN(1,STATUS='UNKNOWN',FILE='DATFIL')
T=0.
S=0.
H=.001
WHILE(YT>=0.)
    XTOLD=XT
    XTDOLD=XTD
    YTOLD=YT
    YTDOLD=YTD
    XTNOMOLD=XTNOM
    XTDNOMOLD=XTDNOM
    YTNOMOLD=YTNOM
    YTDNOMOLD=YTDNOM
    XTDD=0.
    YTDD=-G
    XTDDNOM=0.
    YTDDNOM=-G
    XT=XT+H*XTD
    XTD=XTD+H*XTDD
    YT=YT+H*YTD
    YTD=YTD+H*YTDD
    XTNOM=XTNOM+H*XTDNOM
    XTDNOM=XTDNOM+H*XTDDNOM
    YTNOM=YTNOM+H*YTDNOM
    YTDNOM=YTDNOM+H*YTDDNOM
    T=T+H
    XTDD=0.
    YTDD=-G
    XTDDNOM=0.
    YTDDNOM=-G

```

(continued)

Listing 13.3 (Continued)

```

XT=.5*(XTOLD+XT+H*XTD)
XTD=.5*(XTDOLD+XTD+H*XTDD)
YT=.5*(YTOLD+YT+H*YTD)
YTD=.5*(YTDOLD+YTD+H*YTDD)
XTNOM=.5*(XTNOMOLD+XTNOM+H*XTDNOM)
XTDNOM=.5*(XTDNOMOLD+XTDNOM+H*XTDDNOM)
YTNOM=.5*(YTNOMOLD+YTNOM+H*YTDNOM)
YTDNOM=.5*(YTDNOMOLD+YTDNOM+H*YTDDNOM)
S=S+H
IF(S>=(TS-.00001))THEN
    S=0.
    RTNOM=SQRT((XTNOM-XR)**2+(YTNOM-YR)**2)
    THET=ATAN2((YT-YR),(XT-XR))
    RT=SQRT((XT-XR)**2+(YT-YR)**2)
    THETNOM=ATAN2((YTNOM-YR),(XTNOM-XR))
    DELTHET=57.3*(THET-THETNOM)
    DELRT=RT-RTNOM
    DELXT=XT-XTNOM
    DELXTD=XTD-XTDNOM
    DELYT=YT-YTNOM
    DELYTD=YTD-YTDNOM
    WRITE(9,*)T,DELXT,DELXTD,DELYT,DELYTD,DELTHET,
1      DELRT
    WRITE(1,*)T,DELXT,DELXTD,DELYT,DELYTD,DELTHET,
1      DELRT
    ENDIF
END DO
PAUSE
CLOSE(1)
END

```

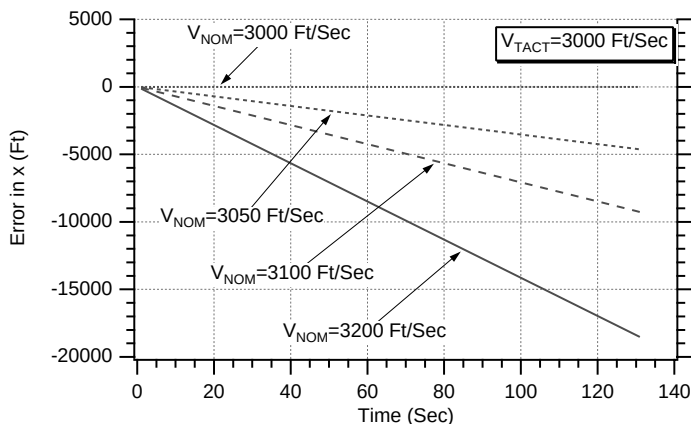


Fig. 13.14 Differences between actual and nominal initial projectile velocities can yield significant differences between actual and nominal downrange of the projectile.

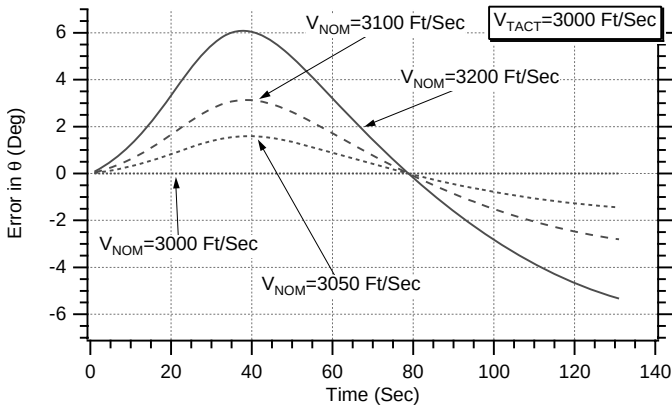


Fig. 13.15 Differences between actual and nominal initial projectile velocities can yield significant differences between actual and nominal angles from the radar to the projectile.

that the error states for the linearized Kalman filter used to track the projectile should be

$$\Delta x = \begin{bmatrix} \Delta x_T \\ \Delta \dot{x}_T \\ \Delta y_T \\ \Delta \dot{y}_T \end{bmatrix} = \begin{bmatrix} x_T - x_{TNOM} \\ \dot{x}_T - \dot{x}_{TNOM} \\ y_T - y_{TNOM} \\ \dot{y}_T - \dot{y}_{TNOM} \end{bmatrix}$$

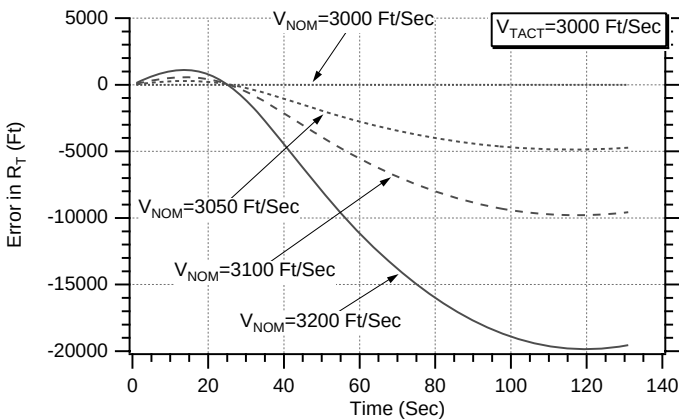


Fig. 13.16 Differences between actual and nominal initial projectile velocities can yield significant differences between actual and nominal ranges from the radar to the projectile.

When the preceding Cartesian error states are chosen, the state-space error differential equation describing projectile motion is linear and becomes

$$\begin{bmatrix} \Delta \dot{x}_T \\ \Delta \ddot{x}_T \\ \Delta \dot{y}_T \\ \Delta \ddot{y}_T \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta x_T \\ \Delta \dot{x}_T \\ \Delta y_T \\ \Delta \dot{y}_T \end{bmatrix} + \begin{bmatrix} 0 \\ u_s \\ 0 \\ u_s \end{bmatrix}$$

In the preceding error equation gravity g is missing because it is assumed to be known exactly and therefore cannot cause any errors. We also have added process noise u_s to the acceleration portion of the error equations as protection for effects that may not be considered by the Kalman filter. However, in our initial experiments we will assume the process noise to be zero because our state-space equations upon which the Kalman filter is based will initially be a perfect match to the real world.

From the preceding state-space equation we can see that systems dynamics matrix is given by

$$\mathbf{F} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

The fundamental matrix can be found from the Taylor-series approximation, which is given by

$$\Phi(t) = \mathbf{I} + \mathbf{F}t + \frac{\mathbf{F}^2 t^2}{2!} + \frac{\mathbf{F}^3 t^3}{3!} + \dots$$

Because in this example

$$\mathbf{F}^2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

all of the higher-order terms must be zero, and the fundamental matrix becomes

$$\Phi(t) = \mathbf{I} + \mathbf{F}t = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} t$$

or more simply

$$\Phi(t) = \begin{bmatrix} 0 & t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & t \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Therefore, the discrete fundamental matrix can be found by substituting the sample time T_s for time t and is given by

$$\Phi_k = \begin{bmatrix} 0 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & T_s \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Because our states have been chosen to be Cartesian, the radar error measurements Δr and $\Delta\theta$ will automatically be nonlinear functions of those states. Therefore, we must write the measurement equation as

$$\begin{bmatrix} \Delta\theta^* \\ \Delta r^* \end{bmatrix} = \begin{bmatrix} \frac{\partial\theta}{\partial x_T} & \frac{\partial\theta}{\partial \dot{x}_T} & \frac{\partial\theta}{\partial y_T} & \frac{\partial\theta}{\partial \dot{y}_T} \\ \frac{\partial r}{\partial x_T} & \frac{\partial r}{\partial \dot{x}_T} & \frac{\partial r}{\partial y_T} & \frac{\partial r}{\partial \dot{y}_T} \end{bmatrix} \begin{bmatrix} \Delta x_T \\ \Delta \dot{x}_T \\ \Delta y_T \\ \Delta \dot{y}_T \end{bmatrix} + \begin{bmatrix} v_\theta \\ v_r \end{bmatrix}$$

where v_θ and v_r represent the measurement noise on the error angle and error range, respectively. Because the angle from the radar to the projectile is given by

$$\theta = \tan^{-1} \left(\frac{y_T - y_R}{x_T - x_R} \right)$$

the four partial derivatives of the angle with respect to each of the states can be computed as

$$\frac{\partial\theta}{\partial x_T} = \frac{1}{1 + (y_T - y_R)^2 / (x_T - x_R)^2} \frac{(x_T - x_R)^* 0 - (y_T - y_R)^* 1}{(x_T - x_R)^2} = \frac{-(y_T - y_R)}{r^2}$$

$$\frac{\partial\theta}{\partial \dot{x}_T} = 0$$

$$\frac{\partial\theta}{\partial y_T} = \frac{1}{1 + (y_T - y_R)^2 / (x_T - x_R)^2} \frac{(x_T - x_R)^* 1 - (y_T - y_R)^* 0}{(x_T - x_R)^2} = \frac{(x_T - x_R)}{r^2}$$

$$\frac{\partial\theta}{\partial \dot{y}_T} = 0$$

Similarly, because the range from the radar to the projectile is given by

$$r = \sqrt{(x_T - x_R)^2 + (y_T - y_R)^2}$$

the four partial derivatives of the range with respect to each of the states can be computed as

$$\frac{\partial r}{\partial x_T} = \frac{1}{2} [(x_T - x_R)^2 + (y_T - y_R)^2]^{-1/2} 2(x_T - x_R) = \frac{(x_T - x_R)}{r}$$

$$\frac{\partial r}{\partial \dot{x}_T} = 0$$

$$\frac{\partial r}{\partial y_T} = \frac{1}{2} [(x_T - x_R)^2 + (y_T - y_R)^2]^{-1/2} 2(y_T - y_R) = \frac{(y_T - y_R)}{r}$$

$$\frac{\partial r}{\partial \dot{y}_T} = 0$$

Because the measurement matrix is given by

$$\mathbf{H} = \begin{bmatrix} \frac{\partial \theta}{\partial x_T} & \frac{\partial \theta}{\partial \dot{x}_T} & \frac{\partial \theta}{\partial y_T} & \frac{\partial \theta}{\partial \dot{y}_T} \\ \frac{\partial r}{\partial x_T} & \frac{\partial r}{\partial \dot{x}_T} & \frac{\partial r}{\partial y_T} & \frac{\partial r}{\partial \dot{y}_T} \end{bmatrix}$$

we can use substitution of the partial derivatives already derived to yield the measurement matrix for this example and obtain

$$\mathbf{H} = \begin{bmatrix} \frac{-(y_T - y_R)}{r^2} & 0 & \frac{x_T - x_R}{r^2} & 0 \\ \frac{x_T - x_R}{r} & 0 & \frac{y_T - y_R}{r} & 0 \end{bmatrix}$$

For this problem it is assumed that we know where the radar is so that x_R and y_R are known and do not have to be estimated. The discrete measurement matrix will be based on the nominal trajectory states or

$$\mathbf{H}_k = \begin{bmatrix} \frac{-(y_{TNOM} - y_R)}{r_{NOM}^2} & 0 & \frac{x_{TNOM} - x_R}{r_{NOM}^2} & 0 \\ \frac{x_{TNOM} - x_R}{r_{NOM}} & 0 & \frac{y_{TNOM} - y_R}{r_{NOM}} & 0 \end{bmatrix}$$

Because the discrete measurement noise matrix is given by

$$\mathbf{R}_k = E(\mathbf{v}_k \mathbf{v}_k^T)$$

we can say that for this problem the discrete measurement noise matrix is

$$\mathbf{R}_k = \begin{bmatrix} \sigma_\theta^2 & 0 \\ 0 & \sigma_r^2 \end{bmatrix}$$

where σ_θ^2 and σ_r^2 are the variances of the angle noise and range noise measurements, respectively. Similarly because the continuous process-noise matrix is given by

$$\mathbf{Q} = E(\mathbf{w} \mathbf{w}^T)$$

we can easily show from the linear state-space equation for this example that the continuous process-noise matrix is

$$\mathbf{Q}(t) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & \Phi_s & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \Phi_s \end{bmatrix}$$

where Φ_s is the spectral density of the white noise sources assumed to be on the downrange and altitude accelerations acting on the projectile. The discrete process-noise matrix can be derived from the continuous process-noise matrix according to

$$\mathbf{Q}_k = \int_0^{T_s} \Phi(\tau) \mathbf{Q} \Phi^T(\tau) d\tau$$

Therefore, substitution of the appropriate matrices into the preceding expression yields

$$\mathbf{Q}_k = \int_0^{T_s} \begin{bmatrix} 1 & \tau & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \tau \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & \Phi_s & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \Phi_s \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ \tau & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \tau & 1 \end{bmatrix} d\tau$$

After some algebra we get

$$\mathbf{Q}_k = \int_0^{T_s} \begin{bmatrix} \tau^2 \Phi_s & \tau \Phi_s & 0 & 0 \\ \tau \Phi_s & \Phi_s & 0 & 0 \\ 0 & 0 & \tau^2 \Phi_s & \tau \Phi_s \\ 0 & 0 & \tau \Phi_s & \Phi_s \end{bmatrix} d\tau$$

Finally, after integration we obtain the final expression for the discrete process-noise matrix to be

$$\mathbf{Q}_k = \begin{bmatrix} \frac{T_s^3 \Phi_s}{3} & \frac{T_s^2 \Phi_s}{2} & 0 & 0 \\ \frac{T_s^2 \Phi_s}{2} & T_s \Phi_s & 0 & 0 \\ 0 & 0 & \frac{T_s^3 \Phi_s}{3} & \frac{T_s^2 \Phi_s}{2} \\ 0 & 0 & \frac{T_s^2 \Phi_s}{2} & T_s \Phi_s \end{bmatrix}$$

We now have defined all of the matrices required to solve the Riccati equations. The equation for the linearized Kalman filter is given by

$$\begin{bmatrix} \Delta \hat{\mathbf{x}}_{T_k} \\ \Delta \hat{\mathbf{x}}_{T_k} \\ \Delta \hat{\mathbf{y}}_{T_k} \\ \Delta \hat{\mathbf{y}}_{T_k} \end{bmatrix} = \Phi \begin{bmatrix} \Delta \hat{\mathbf{x}}_{T_{k-1}} \\ \Delta \hat{\mathbf{x}}_{T_{k-1}} \\ \Delta \hat{\mathbf{y}}_{T_{k-1}} \\ \Delta \hat{\mathbf{y}}_{T_{k-1}} \end{bmatrix} + \begin{bmatrix} K_{11k} & K_{12k} \\ K_{21k} & K_{22k} \\ K_{31k} & K_{32k} \\ K_{41k} & K_{42k} \end{bmatrix} \begin{bmatrix} \Delta \theta^* \\ \Delta r^* \end{bmatrix} - \mathbf{H} \Phi \begin{bmatrix} \Delta \hat{\mathbf{x}}_{T_{k-1}} \\ \Delta \hat{\mathbf{x}}_{T_{k-1}} \\ \Delta \hat{\mathbf{y}}_{T_{k-1}} \\ \Delta \hat{\mathbf{y}}_{T_{k-1}} \end{bmatrix}$$

where the measurement $\mathbf{H}$ and fundamental Φ matrices have been defined already.

The preceding equations for the Kalman filter and Riccati equations were programmed and are shown in Listing 13.4, along with a simulation of the real world and nominal trajectory. We can see that the process-noise matrix is set to zero in this example (i.e., $\Phi_s = 0$). In addition, we have initially zero nominal errors (i.e., $\text{VTERR} = 0$, $\text{GAMDEGERR} = 0$), which means that the nominal trajectory is a perfect match to the actual trajectory. The initial covariance matrix has the same errors as the one in Chapter 9.

The nominal case of Listing 13.4 was run, and the errors in the estimates of the downrange, downrange velocity, altitude, and altitude velocity are displayed in Figs 13.17–13.20. We can see that because the single-run results fall within the theoretical error bounds approximately 68% of the time then we can say that the linearized Kalman filter appears to be working correctly.

Three other cases were run in which an error was introduced to the nominal velocity. We can see from Fig. 13.21 that if the nominal velocity error is only 50 ft/s or 1.7% the single-run results lie within the theoretical error bounds approximately 68% of the time, indicating that the linearized Kalman filter still appears to be working correctly. However, if the velocity error is increased to 100 ft/s or 3.3%, Fig. 13.22 indicates that the single-run error in the estimate of downrange starts to drift outside the error bounds near the end of the run. This means that the linearized filter is beginning to have problems. Finally, Fig. 13.23 shows that when the nominal velocity is in error by 200 ft/s or 6.7% there is total disagreement between the single-run errors in the estimates of downrange and the

Listing 13.4 Linearized Kalman filter for tracking cannon-launched projectile

```

C THE FIRST THREE STATEMENTS INVOKE THE ABSOFT RANDOM
  NUMBER GENERATOR ON THE MACINTOSH
  GLOBAL DEFINE
    INCLUDE 'quickdraw.inc'
  END
  IMPLICIT REAL*8(A-H,O-Z)
  REAL*8 P(4,4),Q(4,4),M(4,4),PHI(4,4),HMAT(2,4),HT(4,2),PHIT(4,4)
  REAL*8 RMAT(2,2),IDN(4,4),PHIP(4,4),PHIPPHIT(4,4),HM(2,4)
  REAL*8 HMHT(2,2),HMHTR(2,2),HMHTRINV(2,2),MHT(4,2),K(4,2)
  REAL*8 KH(4,4),IKH(4,4),XH(4,1),MEAS(2,1),PHIXH(4,1),HPHIXH(2,1)
  REAL*8 RES(2,1),KRES(4,1)
  INTEGER ORDER
  TS=1.
  ORDER=4
  PHIS=0.
  SIGTH=.01
  SIGR=100.
  VT=3000.
  VTERR=0.
  GAMDEG=45.
  GAMDEGERR=0.
  VTNOM=VT+VTERR
  GAMDEGNOM=GAMDEG+GAMDEGERR
  G=32.2
  XT=0.
  YT=0.
  XTD=VT*COS(GAMDEG/57.3)
  YTD=VT*SIN(GAMDEG/57.3)
  XTNOM=XT
  YTNOM=YT
  XTDNOM=VTNOM*COS(GAMDEGNOM/57.3)
  YTDNOM=VTNOM*SIN(GAMDEGNOM/57.3)
  XR=100000.
  YR=0.
  OPEN(1,STATUS='UNKNOWN',FILE='DATFIL')
  OPEN(2,STATUS='UNKNOWN',FILE='COVFIL')
  T=0.
  S=0.
  H=.001
  XH(1,1)=XT-XTNOM
  XH(2,1)=XTD-XTDNOM
  XH(3,1)=YT-YTNOM
  XH(4,1)=YTD-YTDNOM
  DO 14 I=1,ORDER
  DO 14 J=1,ORDER
  PHI(I,J)=0.
  P(I,J)=0.
  Q(I,J)=0.
  IDN(I,J)=0.

```

(continued)

Listing 13.4 (Continued)

```

14 CONTINUE
  TS2=TS*TS
  TS3=TS2*TS
  Q(1,1)=PHIS*TS3/3.
  Q(1,2)=PHIS*TS2/2.
  Q(2,1)=Q(1,2)
  Q(2,2)=PHIS*TS
  Q(3,3)=Q(1,1)
  Q(3,4)=Q(1,2)
  Q(4,3)=Q(3,4)
  Q(4,4)=Q(2,2)
  PHI(1,1)=1.
  PHI(1,2)=TS
  PHI(2,2)=1.
  PHI(3,3)=1.
  PHI(3,4)=TS
  PHI(4,4)=1.
  CALL MATTRN(PHI,ORDER,ORDER,PHIT)
  RMAT(1,1)=SIGTH**2
  RMAT(1,2)=0.
  RMAT(2,1)=0.
  RMAT(2,2)=SIGR**2
  IDN(1,1)=1.
  IDN(2,2)=1.
  IDN(3,3)=1.
  IDN(4,4)=1.
  P(1,1)=1000.**2
  P(2,2)=100.**2
  P(3,3)=1000.**2
  P(4,4)=100.**2
  WHILE(YT>=0.)
    XTOLD=XT
    XTDOLD=XTD
    YTOLD=YT
    YTDOLD=YTD
    XTNOMOLD=XTNOM
    XTDNOMOLD=XTDNOM
    YTNOMOLD=YTNOM
    YTDNOMOLD=YTDNOM
    XTDD=0.
    YTDD=-G
    XTDDNOM=0.
    YTDDNOM=-G
    XT=XT+H*XTD
    XTD=XTD+H*XTDD
    YT=YT+H*YTD
    YTD=YTD+H*YTDD
    XTNOM=XTNOM+H*XTDNOM
    XTDNOM=XTDNOM+H*XTDDNOM
    YTNOM=YTNOM+H*YTDNOM

```

(continued)

Listing 13.4 (Continued)

```

YTDNOM=YTDNOM+H*YTDDNOM
T=T+H
XTDD=0.
YTDD=-G
XTDDNOM=0.
YTDDNOM=-G
XT=.5*(XTOLD+XT+H*XTD)
XTD=.5*(XTDOLD+XTD+H*XTDD)
YT=.5*(YTOLD+YT+H*YTD)
YTD=.5*(YTDOLD+YTD+H*YTDD)
XTNOM=.5*(XTNOMOLD+XTNOM+h*XTDNOM)
XTDNOM=.5*(XTDNOMOLD+XTDNOM+H*XTDDNOM)
XTNOM=.5*(YTNOMOLD+YTNOM+H*YTDNOM)
YTDNOM=.5*(YTDNOMOLD+YTDNOM+H*YTDDNOM)
S=S+H
IF(S>=(TS-.00001))THEN
    S=0.
    RTNOM=SQRT((XTNOM-XR)**22+(YTNOM-YR)**2)
    HMAT(1,1)=- (YTNOM-YR)/RTNOM**2
    HMAT(1,2)=0.
    HMAT(1,3)=(XTNOM-YR)/RTNOM**2
    HMAT(1,4)=0.
    HMAT(2,1)=(XTNOM-XR)/RTNOM
    HMAT(2,2)=0.
    HMAT(2,3)=(YTNOM-YR)/RTNOM
    HMAT(2,4)=0.
    CALL MATTRN(HMAT,2,ORDER,HT)
    CALL MATMUL(PHI,ORDER,ORDER,P,ORDER,ORDER,
        PHIP)
    CALL MATMUL(PHIP,ORDER,ORDER,PHIT,ORDER,
        ORDER,PHIPPHIT)
    CALL MATADD(PHIPPHIT,ORDER,ORDER,Q,M)
    CALL MATMUL(HMAT,2,ORDER,M,ORDER,ORDER,HM)
    CALL MATMUL(HM,2,ORDER,HT,ORDER,2,HMHT)
    CALL MATADD(HMHT,ORDER,ORDER,RMAT,HMHTR)
    DET=HMHTR(1,1)*HMHTR(2,2)-HMHTR(1,2)*HMHTR(2,1)
    HMHTRINV(1,1)=HMHTR(2,2)/DET
    HMHTRINV(1,2)=-HMHTR(1,2)/DET
    HMHTRINV(2,1)=-HMHTR(2,1)/DET
    HMHTRINV(2,2)=HMHTR(1,1)/DET
    CALL MATMUL(M,ORDER,ORDER,HT,ORDER,2,MHT)
    CALL MATMUL(MHT,ORDER,2,HMHTRINV,2,2,K)
    CALL MATMUL(K,ORDER,2,HMAT,2,ORDER,KH)
    CALL MATSUB(IDN,ORDER,ORDER,KH,IKH)
    CALL MATMUL(IKH,ORDER,ORDER,M,ORDER,
        ORDER,P)
    CALL GAUSS(THETNOISE,SIGTH)
    CALL GAUSS(RTNOISE,SIGR)
    THET=ATAN2((YT-YR),(XT-XR))

```

(continued)

Listing 13.4 (Continued)

```

RT=SQRT((XT-XR)**2+(YT-YR)**2)
THETNOM=ATAN2((YT-NOM-YR),(XTNOM-XR))
RTNOM=SQRT((XTNOM-XR)**2+(YT-NOM-YR)**2)
DELTHET=THET-THETNOM
DELRT=RT-RTNOM
MEAS(1,1)=DELTHET+THETNOISE
MEAS(2,1)=DELRT+RTNOISE
CALL MATMUL(PHI,ORDER,ORDER,XH,ORDER,1,
             PHIXH)
CALL MATMUL(HMAT,2,ORDER,PHIXH,ORDER,1,
             HPHIXH)
CALL MATSUB(MEAS,2,1,HPHIXH,RES)
CALL MATMUL(K,ORDER,2,RES,2,1,KRES)
CALL MATADD(PHIXH,ORDER,1,KRES,XH)
XTH=XTNOM+XH(1,1)
XTDH=XTDNOM+XH(2,1)
YTH=YT-NOM+XH(3,1)
YTDH=YTDNOM+XH(4,1)
ERRX=XT-XTH
SP11=SQRT(P(1,1))
ERRXD=XTD-XTDH
SP22=SQRT(P(2,2))
ERRY=YT-YTH
SP33=SQRT(P(3,3))
ERRYD=YTD-YTDH
SP44=SQRT(P(4,4))
WRITE(9,*)T,XT,YT,THET*57.3,RT
WRITE(1,*)T,XT,XTH,XTD,XTDH,YT,YTH,YTD,YTDH
WRITE(2,*)T,ERRX,SP11,-SP11,ERRXD,SP22,-SP22,
1      ERRY,SP33,-SP33,ERRYD,SP44,-SP44
      ENDIF
    END DO
  PAUSE
  CLOSE(1)
  CLOSE(2)
END

```

```

C SUBROUTINE GAUSS IS SHOWN IN LISTING 1.8
C SUBROUTINE MATTRN IS SHOWN IN LISTING 1.3
C SUBROUTINE MATMUL IS SHOWN IN LISTING 1.4
C SUBROUTINE MATADD IS SHOWN IN LISTING 1.1
C SUBROUTINE MATSUB IS SHOWN IN LISTING 1.2

```

theoretical predictions of the covariance matrix. This type of behavior indicates that the linearized Kalman filter cannot tolerate this large of a discrepancy between the nominal and actual initial projectile velocities. A small error in the nominal initial velocity causes a large trajectory error and is thus able to ruin the effectiveness of the extended Kalman filter. Similar results were obtained when the nominal initial velocity was decreased by 50, 100, and 200 ft/s.

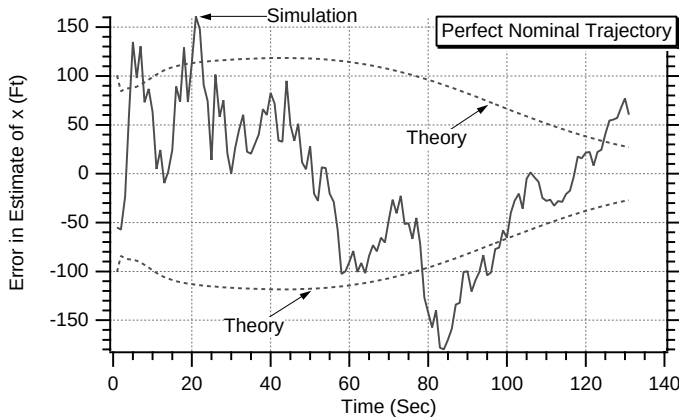


Fig. 13.17 Downrange errors in the estimates are within the theoretical bounds when the nominal trajectory is perfect.

Summary

The messages from the two examples of this chapter are quite clear. When the nominal trajectory, upon which the linearized Kalman filter is based, is accurate, then wonderful estimates can be obtained from the linearized Kalman filter. Of course, if the nominal trajectory were accurate and no error perturbations existed, estimates would not be required because all information could be obtained directly from the nominal trajectory. We also have observed, via simulation experiments, that when the nominal trajectory is inaccurate the estimates from the linearized Kalman filter deteriorate. In fact, significant errors in the nominal trajectory can render the linearized Kalman filter worthless.

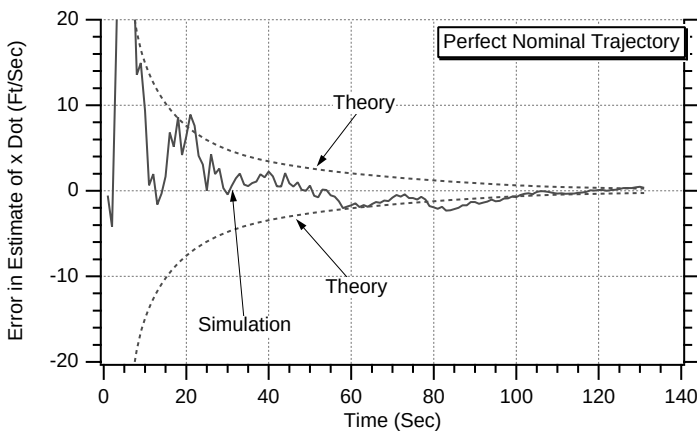


Fig. 13.18 Downrange velocity errors in the estimates are within the theoretical bounds when the nominal trajectory is perfect.

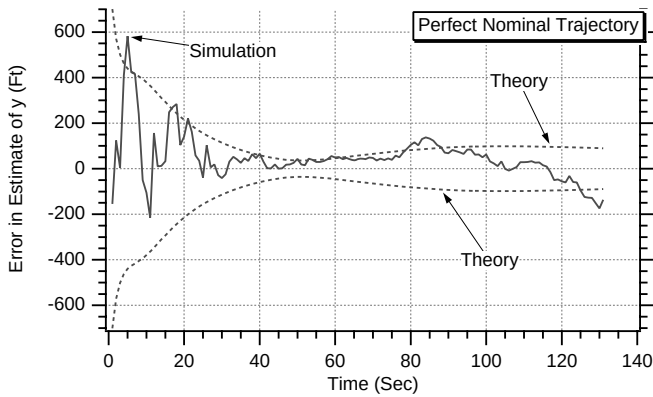


Fig. 13.19 Altitude errors in the estimates are within the theoretical bounds when the nominal trajectory is perfect.

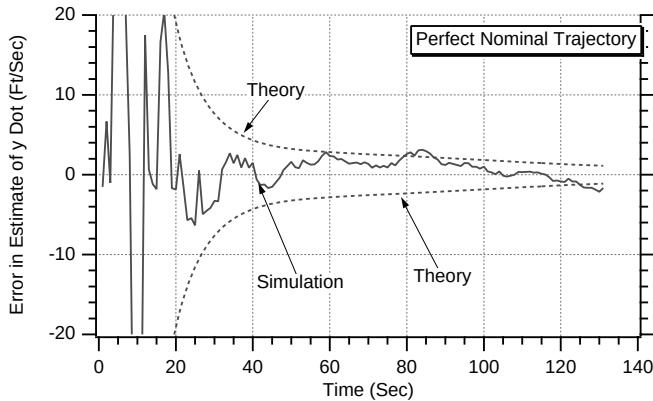


Fig. 13.20 Altitude velocity errors in the estimates are within the theoretical bounds when the nominal trajectory is perfect.

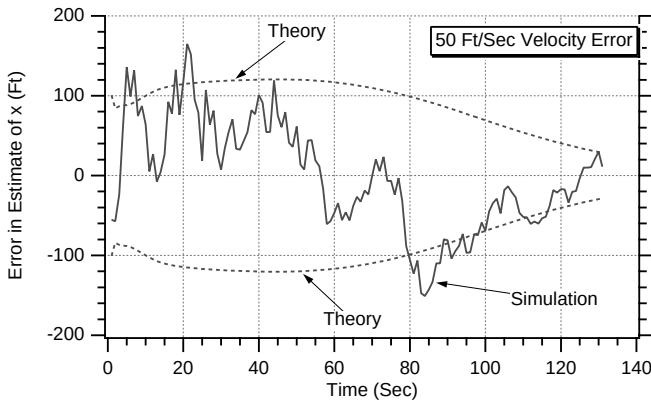


Fig. 13.21 Error in estimate of projectile downrange is between theoretical error bounds when there is a 1.7% error in the nominal velocity.

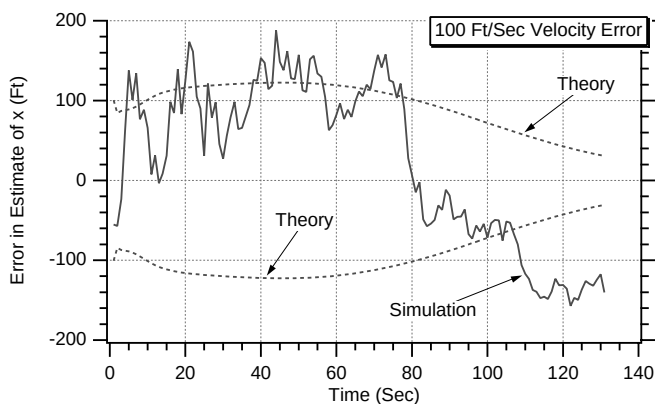


Fig. 13.22 Error in estimate of projectile downrange starts to drift outside theoretical error bounds when there is a 3.3% error in the nominal velocity.

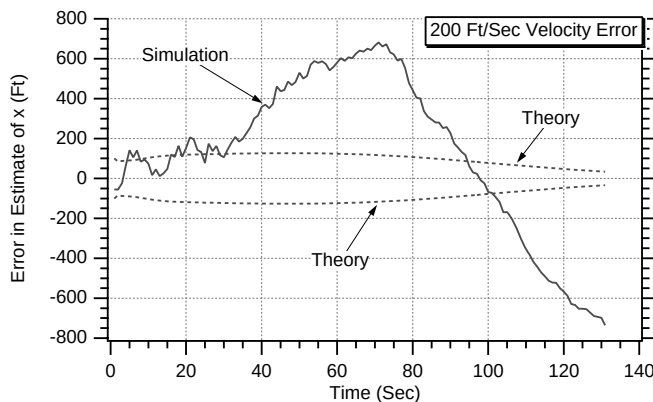


Fig. 13.23 Error in estimate of projectile downrange has no relationship to theoretical error bounds when there is a 6.7% error in the nominal velocity.

References

- ¹Gelb, A., *Applied Optimal Estimation*, Massachusetts Inst. of Technology Press, Cambridge, MA, 1974, pp. 180–190.
- ²Zarchan, P., *Tactical and Strategic Missile Guidance*, 3rd ed., Progress in Astronautics and Aeronautics, AIAA, Reston, VA, 1998, pp. 210–211.

This page intentionally left blank